การนำเสนอโครงงานพัฒนาระบบเว็บแอปพลิเคชัน (Django Framework)

ระบบจัดการร้านอาหารและสั่งอาหารออนไลน์ 'อร่อยเกินเบอร์' (Aroy Gern Ber)

กรณีศึกษา: ร้านผัดกะเพรารวมมิตร ครอบคลุมระบบสั่งอาหาร โต๊ะ QR เดลิเวอรี และคลังสต็อก

เทคโนโลยีหลัก: Python 3.14 · Django 6.0 · Bootstrap 5 · Docker · Render Cloud

URL ใช้งานจริง: https://omg02.onrender.com | **GitHub:** omgx624-gif/OMg02

บัญชีทดสอบ: Admin: admin (Admin1234!) | Staff: staff (Staff1234!) | Customer: customer (User1234!)

ปัญหาและที่มาของโครงงาน (Problem Statement & Motivation)

การบริหารจัดการร้านอาหารแบบดั้งเดิม vs การแก้ปัญหาด้วยเว็บแอปพลิเคชัน

☐ ปัญหาของร้านอาหารแบบดั้งเดิม (Pain Points)

- **การจดออเดอร์ด้วยกระดาษ:** เกิดข้อผิดพลาดง่าย ลายมืออ่านยาก ออเดอร์ตกหล่น
- **การส่งออเดอร์เข้าห้องครัวล่าช้า:** พนักงานต้องเดินไปส่งใบออเดอร์ เสียเวลาบริการลูกค้า
- **ไม่ทราบสต็อกวัตถุดิบเรียลไทม์:** วัตถุดิบหมดกลางคันโดยไม่รู้ล่วงหน้า เสียโอกาสขาย
- **คิดเงินและคำนวณยอดขายช้า:** สรุปยอดขายประจำวันต้องนับบิลกระดาษ ตรวจสอบย้อนหลังยาก
- **การจัดส่งเดลิเวอรีไม่มีระบบ:** จดที่อยู่ผิด พลาดการติดตามสถานะการส่งของไรเดอร์

☐ โซลูชันของระบบ 'อร่อยเกินเบอร์' (Our Solution)

- **สั่งอาหารผ่านเว็บ/QR โต๊ะ:** ลูกค้าสั่งเองได้ทันที ไม่ต้องรอพนักงานมารับรายการ
- **ระบบ Kitchen Display:** ออเดอร์เด้งเข้าห้องครัวแบบเรียลไทม์ พร้อมตัวนับเวลารอ
- **สูตรอาหารตัดสต็อกอัตโนมัติ:** สั่งเมนูไหน วัตถุดิบในคลังลดลงทันที พร้อมเตือนใกล้หมด
- **แดชบอร์ดสรุปยอดขาย + Chart.js:** สถิติรายวัน/สัปดาห์/เดือน พร้อมส่งออก Excel/CSV
- **เดลิเวอรีคำนวณพิกัด GPS อัตโนมัติ:** ปักหมุดที่อยู่ คำนวณระยะทางและค่าส่งตามจริง

วัตถุประสงค์และภาพรวมระบบ (Objectives & Architecture)

เป้าหมายการพัฒนา และสถาปัตยกรรม Model-Template-View (MTV)

☐ วัตถุประสงค์โครงงาน

วัตถุประสงค์หลักของโครงงาน:

1. พัฒนาระบบเว็บแอปพลิเคชันบริหารจัดการร้านอาหารครบวงจรด้วย Django Framework 6.0
2. รองรับกระบวนการสั่งอาหารทั้งหน้าร้าน (Dine-in) และจัดส่งถึงบ้าน (Delivery)
3. ออกแบบระบบควบคุมสิทธิ์ผู้ใช้ 3 บทบาทจริง (Customer, Staff, Admin)
4. พัฒนาฟังก์ชันโบนัสขั้นสูง: REST API, External API, กราฟสถิติ, และ Automated Tests 245 ข้อ
5. ติดตั้งใช้งานจริงบน Cloud Platform (Render.com) ผ่าน Docker Containerization

☐☐ สถาปัตยกรรม Django MTV

สถาปัตยกรรม MTV (Model-Template-View):

- **Model (Data & Logic):** myapp/models.py — ควบคุมตารางฐานข้อมูลและกฎความสัมพันธ์
- **Template (Presentation):** myapp/templates/ — หน้าจอ UI พัฒนาด้วย Bootstrap 5
- **View (Business Controller):** myapp/views/*.py — รับคำขอ ตรวจสิทธิ์ และประมวลผลตรรกะ
- **URLs Router:** myproject/urls.py — กำหนดเส้นทางเข้าถึงแต่ละโมดูลอย่างเป็นสัดส่วน
- **Service Layer:** myapp/services.py — เชื่อมต่อภายนอก (Google Maps, LINE, Nominatim)

โครงสร้างฐานข้อมูล & ความสัมพันธ์ Model (Entity Relationships)

ออกแบบความสัมพันธ์เชิงสัมพันธ์ครบทั้ง 3 รูปแบบตามทฤษฎีฐานข้อมูล

รูปแบบความสัมพันธ์	คู่ความสัมพันธ์ในระบบ	Model ที่เกี่ยวข้อง	ความสำคัญทางธุรกิจ
One-to-One (1:1)	User <--> Profile	django.auth.User <-> Profile	ขยายข้อมูลผู้ใช้: เก็บรูปโปรไฟล์, ปก, คะแนนสะสม, สถานะสมาชิก
One-to-Many (1:N)	Category --> MenuItem	Category (1) -> MenuItem (N)	หมวดหมู่อาหาร 1 หมวด บรรจุเมนูอาหารได้หลายเมนู
One-to-Many (1:N)	Order --> OrderItem	Order (1) -> OrderItem (N)	ออเดอร์ 1 ครั้ง สั่งอาหารได้หลายรายการ พร้อมนับจำนวนและราคา
One-to-Many (1:N)	User --> Order	User (1) -> Order (N)	ลูกค้า 1 บัญชี สามารถดูประวัติการสั่งอาหารย้อนหลังทั้งหมดได้
Many-to-Many (N:M)	MenuItem <--> Ingredient	MenuItem <-> MenuIngredient <-> Ingredient	สูตรอาหาร (Recipe): 1 เมนูใช้วัตถุดิบหลายตัว ใช้ตัดสต็อกอัตโนมัติ
Many-to-Many (N:M)	OrderItem <--> AddOn	OrderItem <-> AddOn (ManyToManyField)	รายการที่สั่ง 1 จาน สามารถเลือกเครื่องเคียงได้หลายรายการ

บทบาทและสิทธิ์ผู้ใช้งาน (User Roles & Permissions Matrix)

การกำหนดสิทธิ์ 3 บทบาทจริง ป้องกันความปลอดภัยด้วย Decorator & Mixins

โมดูล / ฟังก์ชันการทำงาน	ลูกค้า (Customer)	พนักงาน (Staff)	ผู้ดูแลระบบ (Admin)
ดูเมนูอาหาร, ค้นหา, กรองหมวดหมู่	☐ เข้าชมได้	☐ เข้าชมได้	☐ เข้าชมได้
สั่งอาหาร, เลือกเครื่องเคียง, แนบสลิป	☐ สั่งได้	☐ สั่งแทนลูกค้าได้	☐ สั่งได้
ดูประวัติออเดอร์ของตนเอง	☐ ดูได้เฉพาะของตนเอง	☐ ดูได้ทั้งหมด	☐ ดูได้ทั้งหมด
เพิ่ม / แก้ไข / ลบ เมนูอาหาร (CRUD)	☐ ไม่มีสิทธิ์ (403)	☐ จัดการได้	☐ จัดการได้
จัดการสต็อกวัตถุดิบ & ปรับปรุงคลัง	☐ ไม่มีสิทธิ์ (403)	☐ จัดการสต็อกได้	☐ จัดการได้
Kitchen Display & ปรับสถานะทำอาหาร	☐ ไม่มีสิทธิ์ (403)	☐ จัดการได้	☐ จัดการได้
Dashboard ยอดขาย & ส่งออก Excel/CSV	☐ ไม่มีสิทธิ์ (403)	☐ ดูสถิติ/Export ได้	☐ ดูสถิติ/Export ได้
เข้าใช้งาน Django Admin (/admin/)	☐ ไม่มีสิทธิ์ (บล็อก)	☐ ไม่มีสิทธิ์	☐ สิทธิ์เต็ม (Superuser)

ตรวจสอบเกณฑ์ขั้นต่ำ 10 ข้อ (Core Requirements Checklist)

ระบบผ่านเกณฑ์ขั้นต่ำของโจทย์ครบถ้วน 100% ทุกข้อ

#	เกณฑ์ที่กำหนดตามโจทย์	ฟังก์ชันในระบบ 'อร่อยเกินเบอร์'	สถานะ
1	**ระบบสมัครสมาชิก เข้าสู่ระบบ และออกจากระบบ**	Register, Login, Logout, รองรับ Google OAuth 2.0 (django-allauth)	☐ ผ่าน 100%
2	**ผู้ใช้งานอย่างน้อย 2 บทบาท พร้อมกำหนดสิทธิ์จริง**	3 บทบาท: Customer, Staff, Admin ตรวจสอบด้วย @login_required, @user_passes_test	☐ ผ่าน 100%
3	**การเพิ่ม แสดง แก้ไข ลบข้อมูล (CRUD) ด้วย Model**	CRUD เมนูอาหาร, หมวดหมู่, ออเดอร์, วัตถุดิบ, คูปอง ผ่าน Django ModelForm	☐ ผ่าน 100%
4	**ระบบค้นหาข้อมูล (Search & Filter)**	ค้นหาชื่อเมนู, กรองตามหมวดหมู่, กรองช่วงราคา, ระดับความเผ็ด และค้นหาออเดอร์	☐ ผ่าน 100%
5	**ระบบอัปโหลดรูปภาพหรือไฟล์**	อัปโหลดรูปเมนูอาหาร, รูปโปรไฟล์ และสลิป พร้อมตัวตรวจสอบไฟล์ <= 10MB ด้วย Pillow	☐ ผ่าน 100%
6	**การแบ่งหน้ารายการข้อมูล (Pagination)**	ใช้ Django Paginator ในหน้ารายการอาหาร (menu_list) และประวัติออเดอร์	☐ ผ่าน 100%
7	**ความสัมพันธ์ระหว่าง Model อย่างน้อย 1 รูปแบบ**	มีครบทั้ง 3 แบบ: One-to-One, One-to-Many, และ Many-to-Many	☐ ผ่าน 100%
8	**การตรวจสอบความถูกต้องของข้อมูลในฟอร์ม**	Form Validation: ราคา > 0, เลขโต๊ะ 1-999, ชื่อไม่ซ้ำ, นามสกุลและขนาดไฟล์	☐ ผ่าน 100%
9	**UI มีรูปแบบสม่ำเสมอ อ่านง่าย ใช้งานได้จริง**	พัฒนาด้วย Bootstrap 5, Font Awesome, Responsive Design ทุกอุปกรณ์	☐ ผ่าน 100%
10	**นำระบบขึ้น Hosting/Cloud ใช้งานผ่านเน็ตได้**	Deploy บน Render.com ผ่าน Dockerfile พร้อมใช้งานที่ https://omg02.onrender.com	☐ ผ่าน 100%

ระบบสั่งอาหาร ใบเสร็จ และเดลิเวอรี (Ordering & Delivery Workflow)

ขั้นตอนการไหลของข้อมูลตั้งแต่ลูกค้าสั่งอาหารจนถึงการส่งมอบ

□□ การสั่งอาหารหน้าร้าน (Dine-in)

- **เปิดโต๊ะ & QR Code:** ลูกค้าเปิดสั่งตามเลขโต๊ะ หรือสแกน QR ประจำโต๊ะ
- **ปรับแต่งรายการ:** เลือกระดับความเผ็ด (0-3) และเพิ่มเครื่องเคียง เช่น ไข่ดาว/กุนเชียง
- **รหัสออเดอร์รายวัน:** สร้างรหัสสั้นแสดงผล เช่น #001 รีเซ็ตใหม่ทุกวัน
- **ใบเสร็จรับเงิน (Receipt):** แสดงยอดรวม ส่วนลดคูปอง และมีปุ่มพิมพ์ใบเสร็จทันที

□ การจัดส่งเดลิเวอรี (Delivery & GPS)

- **บันทึกที่อยู่จัดส่ง:** เก็บชื่อ เบอร์โทร และที่อยู่จัดส่งของผู้สั่ง
- **คำนวณระยะทาง GPS:** ใช้สูตร Haversine คำนวณระยะทางจากพิกัดร้านจริง
- **คำนวณค่าส่งอัตโนมัติ:** คิดอัตราค่าส่งตามกิโลเมตรจริงอย่างเป็นธรรม
- **Rider Dashboard:** ไรเดอร์สามารถกดรับงาน เปลี่ยนสถานะ และโทรติดต่อลูกค้าได้

ฟังก์ชันความสามารถพิเศษ (Bonus Features - สูงสุด 5 คะแนน)

ฟังก์ชันขั้นสูงที่พัฒนาเพิ่มเติมเกินขอบเขตขั้นต่ำ เพื่อเพิ่มคุณภาพของระบบ

1. RESTful API (DRF)

เปิด Endpoint /api/menus/, /api/orders/, /api/categories/ รองรับ CRUD และเชื่อมต่อกับระบบภายนอก

2. External API Integration

- Google Maps Embed (แผนที่ร้าน)
- LINE Notify (แจ้งเตือนออเดอร์เข้ากลุ่มครัว)
- OpenStreetMap (พิกัด GPS ที่อยู่)

3. Dashboard & Chart.js

หน้าแดชบอร์ดสรุปยอดขาย สถิติรายวัน/สัปดาห์/เดือน พร้อมกราฟแท่งแนวโน้มยอดขาย 7 วันย้อนหลัง และจัดอันดับเมนูขายดี

4. Automated Testing (245 Tests)

ชุดทดสอบอัตโนมัติครอบคลุมทั้งระบบจำนวน **245 ข้อ** ผ่าน 100% ตรวจสอบ Model, View, Security, API และ Logic

5. Data Export (Excel & CSV)

ส่งออกรายงานยอดขายและประวัติออเดอร์เป็นไฟล์ Excel (.xlsx) สวยงามด้วย openpyxl และไฟล์ CSV สำหรับนำไปวิเคราะห์ต่อ

6. Docker & Containerization

Dockerfile แบบ Multi-stage Build ปลอดภัย น้ำหนักเบา พร้อมรัน Production บน Render.com อย่างมีเสถียรภาพ

การประกันคุณภาพและความปลอดภัย (Quality Assurance & Security)

ความมั่นคงปลอดภัยตามมาตรฐาน และผลการทดสอบซอฟต์แวร์ 245 ข้อ

□□ มาตรการความปลอดภัยของระบบ (Security)

- **CSRF Protection:** ป้องกันการปลอมแปลงคำขอข้ามไซต์ทุกฟอร์ม
- **Rate Limiting & Throttle:** ป้องกันการยิงสแปมออเดอร์และการ Brute-Force รหัสผ่าน
- **SQL Injection Safe:** ใช้ Django ORM แบบ Parameterized Queries 100%
- **Secure Headers:** รองรับ HSTS, HTTPS Redirect, และ SameSite Cookies
- **File Upload Security:** ตรวจสอบ MIME type และขนาดไฟล์รูปภาพไม่เกิน 10MB

□ ผลการทดสอบอัตโนมัติ (Automated Testing)

- **คำสั่งรัน:** python manage.py test
- **จำนวนเคส: 245 Test Cases**
- **ผลลัพธ์: OK (100% Passed)** โดยใช้เวลารัน 120 วินาที
- **ขอบเขตที่ตรวจ:** Model Constraints, View Permission, Form Validation, API Endpoints, Recipe Stock Deduction, Haversine Distance Calculation

การติดตั้งบน Cloud Hosting (Production Deployment on Render)

ขั้นตอนการนำระบบขึ้นคลาวด์สาธารณะผ่าน Docker Container

☐ โครงสร้าง Dockerfile สำหรับ Production

- **Multi-Stage Build:** ใช้ Python 3.14-slim แยก Stage บิลด์เพื่อลดขนาด Image
- **Non-Root User:** รันแอปพลิเคชันภายใต้ผู้ใช้ appuser เพื่อความปลอดภัย
- **Gunicorn WSGI:** รันเว็บเซิร์ฟเวอร์แบบ Production รองรับการประมวลผลพร้อมกัน
- **WhiteNoise:** บีบอัดและเสิร์ฟไฟล์ Static Files (CSS/JS) โดยตรงอย่างรวดเร็ว

☐☐ การจัดการข้อมูลบน Cloud (Auto-Seed & Migration)

- **Auto Migrate:** รัน migrate --noinput อัตโนมัติทุกครั้งที่บูตคอนเทนเนอร์
- **Auto Seed Data:** เติมข้อมูลเมนูอาหารทั้ง 16 เมนู พร้อมรูปภาพจริงลงฐานข้อมูล
- **Init Test Users:** สร้างบัญชี Admin, Staff, Customer พร้อมรหัสผ่านให้อัตโนมัติ
- **Healthcheck:** มีระบบตรวจสอบสถานะเซิร์ฟเวอร์ผ่าน Endpoint /healthz

ลำดับขั้นตอนการสาธิตระบบจริง (Live Demonstration Flow)

แนวทางการสาธิตฟังก์ชันการทำงาน 8 ขั้นตอนตามเกณฑ์การประเมินของอาจารย์

ขั้นตอน	กิจกรรมที่สาธิต	สิ่งที่แสดงบนหน้าจอ	บทบาทที่ใช้
1. สมัครสมาชิก	**เปิดหน้า /register/ กรอกข้อมูลและสมัคร**	สมัครสำเร็จ และล็อกอินเข้าสู่ระบบทันที	ผู้ใช้ใหม่
2. เข้าสู่ระบบ	**เข้าหน้า /login/ ด้วยบัญชีลูกค้า**	แสดงชื่อผู้ใช้ และคะแนนสะสมบนแถบ Navbar	Customer
3. กำหนดสิทธิ์	**เปรียบเทียบหน้าจอ Customer vs Staff**	Staff เห็นเมนูคลังสต็อก, ออเดอร์, แดชบอร์ด (User ทั่วไปไม่เห็น)	Staff / User
4. CRUD ข้อมูล	**เพิ่มเมนูใหม่, แก้ไขราคา, ปรับสถานะพร้อมขาย**	ข้อมูลเปลี่ยนแปลงในฐานข้อมูลและหน้าเว็บทันที	Staff / Admin
5. ค้นหา & กรอง	**พิมพ์ค้นหา 'กะเพรา' และคลิกเลือกหมวด 'ผัด'**	รายการอาหารกรองตามคำค้นและหมวดหมู่อย่างถูกต้อง	ทุกคน
6. อัปโหลดไฟล์	**แก้ไขเมนูและอัปโหลดรูปภาพอาหารใหม่**	รูปภาพแสดงผลบนการ์ดเมนูและหน้ารายละเอียด	Staff
7. การแบ่งหน้า	**เข้าหน้ารายการเมนูอาหาร กดเปลี่ยนหน้า 1, 2**	Pagination แสดงจำนวนรายการต่อหน้าและปุ่มเลื่อนหน้า	ทุกคน
8. ใช้งาน Hosting	**เปิดเว็บจริง https://omg02.onrender.com**	ระบบตอบสนองรวดเร็ว ใช้งานได้จริงผ่านเครือข่ายอินเทอร์เน็ต	ทุกคน

ข้อมูลสำหรับทดสอบและส่งงาน (Live Access & Test Accounts)

ลิงก์ URL ระบบจริง, QR Code, และตารางบัญชีผู้ใช้สำหรับทดสอบ

☐ **Cloud Hosting URL:**
https://omg02.onrender.com
(เข้าใช้งานระบบจริงผ่านอินเทอร์เน็ต)

☐ **GitHub Repository:**
https://github.com/omgx624-gif/OMg02
(ซอร์สโค้ดโครงงานฉบับสมบูรณ์)

บทบาท (Role)	Username	Password	สิทธิ์และสิ่งที่ทดสอบได้
Admin (ผู้ดูแลระบบ)	admin	Admin1234!	เข้า Django Admin (/admin/), จัดการฐานข้อมูลทั้งหมด, ควบคุมสิทธิ์
Staff (พนักงาน)	staff หรือ Jj12	Staff1234!	ดูแดชบอร์ดร้าน, จัดการออเดอร์, คลังสต็อก, จัดการเดลิเวอรี
Customer (ลูกค้า)	customer หรือ Ff12	User1234!	สั่งอาหารหน้าร้าน, จองโต๊ะ, สั่งเดลิเวอรี, สะสมแต้ม, เขียนรีวิว

รายการเอกสารและสิ่งที่ต้องส่ง (Project Deliverables Checklist)

ความพร้อมของเอกสาร ซอร์สโค้ด และระบบตามข้อกำหนดของโครงงาน

สิ่งที่ต้องส่งตามเกณฑ์	รายละเอียดและไฟล์ที่จัดเตรียม	สถานะ
1. URL ระบบบน Hosting	ระบบออนไลน์บน Render: https://omg02.onrender.com พร้อมใช้งานจริง	☐ เรียบร้อย
2. บัญชีผู้ใช้ทดสอบระบบ	ตาราง Username / Password สำหรับ Admin, Staff, Customer ครบ 3 บทบาท	☐ เรียบร้อย
3. GitHub Repository	ซอร์สโค้ดฉบับสมบูรณ์ที่ https://github.com/omgx624-gif/OMg02	☐ เรียบร้อย
4. เล่มรายงานโครงงาน PDF	รายงานโครงงานฉบับสมบูรณ์ 5 บท + ภาคผนวก ในโฟลเดอร์ docs/ (PDF 1 เล่ม)	☐ เรียบร้อย
5. ไฟล์นำเสนอ (Slide PDF)	สไลด์นำเสนอฉบับทางการ 14 หน้า แนวนอน 16:9 ในโฟลเดอร์ docs/ (PDF 1 อัน)	☐ เรียบร้อย
6. การสาธิตระบบจริง	เตรียมลำดับการสาธิต 8 ขั้นตอน พร้อมบัญชีทดสอบที่กำหนดค่าไว้ล่วงหน้า	☐ พร้อมนำเสนอ

ขอขอบพระคุณคณะกรรมการและอาจารย์ผู้สอน

จบการนำเสนอโครงงาน 'อร่อยเกินเบอร์' (Aroy Gern Ber)

ยินดีรับฟังข้อเสนอแนะและตอบคำถาม (Questions & Answers)

จุดเด่นของระบบ: ฟังก์ชันครบตามเกณฑ์ 10 ข้อ · คะแนน Bonus ครบทุกหมวด · Automated Tests 245 ข้อผ่าน 100% · รันบน Cloud จริง

URL: https://omg02.onrender.com | **GitHub:** https://github.com/omgx624-gif/OMg02